

Tesla Stock Price Prediction

Using SimpleRNN and LSTM Deep Learning Models

Domain: Financial Services

Project Report

Submitted By: Rohan R J

Batch: DS-C-WD-E-B48

Contents

1. Problem Statement
2. Approach and Methodology
3. Data Cleaning and Missing Value Handling
4. Data Preprocessing and Feature Engineering
5. Model Development
6. Hyperparameter Tuning with GridSearchCV
7. Model Training
8. Model Evaluation and Results
9. Insights and Conclusion
10. Limitations and Possible Improvements
11. Business Use Cases

1. Problem Statement

The goal of this project is to build a deep learning model that predicts Tesla's (TSLA) stock closing price. Stock price data is sequential in nature, so the project uses two types of Recurrent Neural Networks: SimpleRNN and LSTM (Long Short-Term Memory), and compares their performance. The model is built to predict the closing price 1 day, 5 days, and 10 days ahead. The dataset used is TSLA.csv, containing daily Open, High, Low, Close, Adjusted Close, and Volume values from 2010 to 2020.

2. Approach and Methodology

The project follows the standard deep learning workflow for time-series forecasting: data loading and cleaning, feature selection, scaling, sequence generation, model building, hyperparameter tuning, training, and evaluation. Both SimpleRNN and LSTM models were built with an identical architecture (aside from the recurrent layer type) so that the comparison between them is fair.

3. Data Cleaning and Missing Value Handling

The dataset was checked for missing values and duplicate dates. No missing values were found in this dataset. However, because this is time-series data (where row order carries meaning), missing values need different handling than in a normal tabular dataset:

- Mean or median fill is not appropriate here, since it would replace a missing day's price with the average price across the entire 10-year period, ignoring whether the stock was trending up or down at that specific point in time.
- Forward-fill (carrying the last known price forward) is the appropriate method, since it is a reasonable estimate for a missing trading day and does not use any future information.
- Backward-fill or interpolation using future values was avoided, since the model must only ever use past information to make predictions - using future values would be a form of data leakage.

4. Data Preprocessing and Feature Engineering

4.0 Exploratory Data Analysis

Before modeling, the data was visualized in several ways: the Close price trend over the full 2010-2020 period, 50-day and 200-day moving averages to smooth out short-term noise and reveal the underlying trend, trading Volume over time, and a correlation heatmap across all price columns. The heatmap confirmed that Open, High, Low, Close, and Adjusted Close are almost perfectly correlated with each other, while Volume is not strongly correlated with price, supporting the choice of Close as the single modeling feature. These charts are included in the Jupyter Notebook.

4.1 Feature Selection

The Close price was selected as the target variable, as instructed by the problem statement. The Date column was converted to a datetime index.

4.2 Scaling

MinMaxScaler was used to scale the Close price into a 0-1 range, which helps the neural network

converge faster during training. The scaler was fit only on the training data and then applied to the test data, to avoid leaking information about future prices into the training process.

4.3 Train/Test Split

The data was split chronologically (not randomly): the first 80% of days (oldest) were used for training, and the last 20% (most recent) were held out as the test set. Shuffling was avoided since it would let the model see future prices during training.

4.4 Sequence Generation

A sliding window approach was used to prepare the data for the RNN/LSTM models: the past 60 days of Close price were used as input to predict the price a given number of days ahead (the prediction horizon). This was repeated for three horizons: 1 day, 5 days, and 10 days.

5. Model Development

5.1 Architecture

Both models use an identical structure, built using the Keras Sequential API:

- A SimpleRNN or LSTM layer, to learn the sequential pattern in the data.
- A Dropout layer, to reduce overfitting by randomly disabling some neurons during training.
- A Dense layer with one output unit, producing the predicted price.

5.2 Compilation

Both models were compiled using Mean Squared Error (MSE) as the loss function and the Adam optimizer.

6. Hyperparameter Tuning with GridSearchCV

GridSearchCV was used to tune three hyperparameters for both the SimpleRNN and LSTM models: the number of units in the recurrent layer, the dropout rate, and the learning rate. Since GridSearchCV is a scikit-learn tool and the models are built with Keras, the KerasRegressor wrapper from the scikeras library was used to make the models compatible with scikit-learn's search interface. Tuning was performed on the 1-day-horizon training data, and the best hyperparameters found were reused for the 5-day and 10-day models of the same architecture.

Model	Units	Dropout Rate	Learning Rate
SimpleRNN	32	0.2	0.01
LSTM	32	0.2	0.01

7. Model Training

Six models in total were trained: SimpleRNN and LSTM, each for the 1-day, 5-day, and 10-day horizons. Each model was trained using the best hyperparameters found through GridSearchCV, with two callbacks:

- EarlyStopping, which halts training once the validation loss stops improving, restoring the best

weights found - this prevents overfitting.

- ModelCheckpoint, which saves the best-performing version of each model to disk during training.

8. Model Evaluation and Results

Each trained model was used to predict prices on the held-out test set. Predictions were converted back from the scaled 0-1 range to real dollar prices, and Mean Squared Error (MSE) was calculated for each model and horizon.

Model	Horizon	MSE
SimpleRNN	1 day	204.31
LSTM	1 day	353.17
SimpleRNN	5 days	725.36
LSTM	5 days	955.41
SimpleRNN	10 days	1482.52
LSTM	10 days	1779.00

Actual versus predicted price charts were also plotted for each horizon (included in the Jupyter Notebook) to visually assess how closely each model's predictions track the real price movements.

9. Insights and Conclusion

- Both models capture the overall trend of Tesla's stock price reasonably well, but tend to lag behind sudden sharp price jumps, since they are trained only on past price data and cannot anticipate news or market events.
- SimpleRNN achieved lower MSE than LSTM at every horizon tested: 204.31 vs 353.17 at 1 day, 725.36 vs 955.41 at 5 days, and 1482.52 vs 1779.00 at 10 days.
- This result goes against the common assumption that LSTM, with its memory cell designed for longer sequences, should always outperform SimpleRNN. With a relatively small dataset (around 1,900 training sequences) and simple single-layer architectures, LSTM's additional parameters gave it more to learn without necessarily more signal to learn from, so the simpler model generalized better here.
- Prediction error increases substantially as the horizon grows, roughly 5-7 times higher at the 10-day horizon compared to the 1-day horizon for both models. This confirms that predicting further into the future is a harder problem, as uncertainty compounds over the longer window.
- Note: neural network training involves randomness (weight initialization, dropout) that is not fully controlled by a single random seed in TensorFlow/Keras, so re-running the notebook can shift the exact MSE values slightly between runs. The overall pattern -- error growing with horizon, and SimpleRNN being competitive with or better than LSTM on this dataset -- has held consistently.

10. Limitations and Possible Improvements

Limitations

- The model uses only the historical Close price as input, so it has no way to anticipate news, earnings announcements, or other market-moving events.
- The MinMaxScaler was fit on training-era prices (2010-2018); since Tesla's price range grew substantially afterward, predictions in that higher price range may be less accurate.
- Hyperparameters were tuned only on the 1-day horizon and reused for the 5-day and 10-day models; tuning separately per horizon could change the results.

Possible Improvements

- Add more input features, such as trading Volume, news sentiment, social media sentiment, or macroeconomic indicators (interest rates, inflation).
- Compare against additional architectures, such as GRU, Transformer-based models, or a classical ARIMA baseline.
- Run GridSearchCV separately for each prediction horizon rather than reusing the 1-day result.
- Apply the same pipeline to competitor stocks (Rivian, NIO, Lucid Motors) for comparative analysis.

11. Business Use Cases

This project's model outputs and evaluation methodology connect to several real-world business use cases:

Stock Market Trading and Investment Strategies

- Automated Trading: the 1-day-ahead prediction (the most accurate horizon in this project) could be used as an input to a simple trading rule, such as buying when the predicted price is higher than the current price. Any such rule would need proper backtesting with transaction costs before real use.
- Risk Management and Portfolio Optimization: the size of the prediction error gives a rough sense of how volatile or unpredictable the stock currently is, which can inform position sizing and hedging decisions.

Financial Forecasting and Time-Series Analysis

- Long-Term Investment Planning: the 5-day and 10-day models provide a medium-term trend signal to support hold or sell decisions, though with higher error than the 1-day model.
- Macroeconomic Analysis: this project used only Close price as input; the same pipeline could be extended with macroeconomic indicators as additional features.

Business and Corporate Use Cases

- Company Valuation and Earnings Prediction: the same windowed sequence-modeling approach could be reused to forecast other financial time series, such as quarterly revenue or earnings per share.
- Competitor Analysis: running this same pipeline on Rivian, NIO, or Lucid Motors data would produce directly comparable MSE figures for cross-company benchmarking.

Deep Learning and Research Use Cases

- Comparing Time-Series Models: this project already compares SimpleRNN and LSTM; the same evaluation approach could be extended to GRU, Transformer models, or ARIMA.
- Feature Engineering and Alternative Data: the current pipeline uses a single input feature (Close

price); adding sentiment or macroeconomic data would only require widening the model's input, without changing the overall architecture.